Studio Nx

Backend(FastAPI/Python) 67 — get_ready_projects()

2026-10-12

Backend(FastAPI/Python) 67 — `get_ready_projects()` (아직 열지 않은 프로젝트)

항목	내용
파일	`studionx/backend/app/services/dashboard_service.py`
작업	P1-16 대시보드 API: 마이페이지의 'Ready to start' 배너 (PRD 01 FR-130)
작성	Claude (AI), 2026-10-12, 사용자 검토
이전 · 다음	66 `get_recent_projects()` · 68 (다음)

1. 하는 일

이 함수는 **내가 시퀀서에서 아직 한 번도 열지 않은** 프로젝트를 최근에 만든 순서대로 최대 3개까지 돌려줍니다. 마이페이지(P1-17) 맨 위에 있는 **'Ready to start'** 배너는 그중 첫 번째 프로젝트를 보여 주고, 나머지 개수는 "1 more in My Projects"처럼 알려 줍니다.

데이터	결과
내가 30일 전에 한 번 열었던 프로젝트	빠집니다 (한 번이라도 열었으면 제외)
새로 만든 프로젝트 4개	최근에 만든 순서로 나옵니다
삭제한 프로젝트, 내가 멤버가 아닌 남의 프로젝트	빠집니다
멤버로 참여한 남의 프로젝트 중 **그 사람은 열었지만 나는 열지 않은 것**	포함됩니다
위 데이터를 모두 합친 경우	`New 0, Shared, New 1` (최대 3개)

작은 화면에서 프로젝트를 만든 사람이 나중에 컴퓨터로 접속하면, 이 배너가 그 프로젝트를 곧바로 보여 줍니다. 이것은 FR-129와 FR-130에 정해진 동작이며, 36번 안내 모달의 문구 "it'll be waiting at the top of your dashboard"가 가리키는 기능이기도 합니다.

2. 언제 쓰나

호출하는 곳	화면
68 `get_dashboard`(예정) → `GET /me/dashboard`	myPage.html의 'Ready to start' 배너 (P1-17)

`NOT EXISTS`(서브쿼리): "이 조건에 맞는 행이 **하나도 없으면**"이라는 뜻을 SQL로 표현하는 방법입니다. 프로젝트마다 "내 세션 행이 있는가?"를 DB 안에서 확인하기 때문에, 파이썬이 세션을 모두 가져와서 걸러 낼 필요가 없습니다. 실제로 DB에 전달되는 SQL은 다음과 같은 모양입니다.

```
SELECT projects.* FROM projects JOIN project_members ON …
WHERE project_members.user_id = 나 AND projects.deleted_at IS NULL
  AND NOT (EXISTS (SELECT session_id FROM project_sessions
                   WHERE project_sessions.project_id = projects.project_id
                     AND project_sessions.user_id = 나))
ORDER BY created_at DESC, project_id DESC LIMIT 3
```

3. 코드

import (파일 맨 위: 66번과 같고, 이번에 새로 추가한 것은 없습니다)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.orm import Session

from app.models import Project, ProjectMember, ProjectSession, User
from app.schemas.dashboard import (
    DashboardAi,
    DashboardResponse,
    DashboardStorage,
    ReadyProject,
    RecentProject,
    SessionItem,
    StorageByKind,
    UsageCount,
)
```

이 함수는 `select`, `Session`, `Project`, `ProjectMember`, `ProjectSession`, `User`, `ReadyProject`와 상수 `READY_PROJECT_COUNT`(3)를 사용합니다.

함수

```
def get_ready_projects(db: Session, user: User) -> list[ReadyProject]:
    opened = (
        select(ProjectSession.session_id)
        .where(
            ProjectSession.project_id == Project.project_id,
            ProjectSession.user_id == user.user_id,
        )
        .exists()
    )
    projects = db.scalars(
        select(Project)
        .join(ProjectMember, ProjectMember.project_id == Project.project_id)
        .where(ProjectMember.user_id == user.user_id, Project.deleted_at.is_(None), ~opened)
        .order_by(Project.created_at.desc(), Project.project_id.desc())
        .limit(READY_PROJECT_COUNT)
    ).all()
    return [
        ReadyProject(
            project_id=p.project_id,
            title=p.title,
            bpm=p.bpm,
            time_sig_num=p.time_sig_num,
            time_sig_den=p.time_sig_den,
            created_at=p.created_at,
        )
        for p in projects
    ]
```

문법 풀이

import 문법	뜻
(새로 추가한 import 없음)	66번에서 추가한 것을 그대로 사용합니다

식	뜻	예
`select(ProjectSession.session_id).where(…).exists()`	"그런 세션 행이 있다"는 조건을 만듭니다 (EXISTS 서브쿼리)	
`ProjectSession.project_id == Project.project_id`	바깥 쿼리가 지금 보고 있는 **그 행**의 프로젝트와 연결합니다 (상관 서브쿼리)	
`~opened`	조건을 뒤집어서 `NOT EXISTS`로 만듭니다	SQLAlchemy에서는 파이썬의 `not` 대신 `~`를 씁니다
`.order_by(Project.created_at.desc(), …)`	최근에 **만든** 순서로 정렬합니다	66번은 최근에 **수정한** 순서입니다
`.limit(READY_PROJECT_COUNT)`	최대 3개까지만 가져옵니다	

4. 흐름

1. "내 세션이 있다"는 조건을 만듭니다.
2. 내가 멤버이고, 삭제되지 않았으며, 그 조건에 해당하지 **않는** 프로젝트를 최근에 만든 순서로 3개까지 가져옵니다.
3. 가져온 프로젝트를 응답 모양(`ReadyProject`)으로 바꿔서 돌려줍니다.

5. 왜 이렇게

선택	이유
`NOT EXISTS`로 쿼리를 **한 번**만 보냄	프로젝트마다 세션을 따로 묻지 않기 위해서입니다. 시험에서 SELECT가 1번만 나가고 `EXISTS`가 들어 있는 것을 확인했습니다 (템플릿 권장 방식)
내 세션만 확인함	58번 `list_projects`의 `opened`와 기준이 같아야, 목록의 `Not opened` 표시와 배너 내용이 서로 어긋나지 않습니다
기간을 제한하지 않음	7일 전이든 1년 전이든 한 번 열었다면 그 프로젝트는 "열어 본 프로젝트"이지, 시작을 기다리는 프로젝트가 아니기 때문입니다. 최근 7일만 보는 66번과 다른 점입니다
최근에 **만든** 순서	방금 만든 프로젝트가 배너 맨 앞에 나와야 하기 때문입니다 (FR-130)
최대 3개	화면은 1개만 보여 주고 나머지는 개수로 알려 주기 때문입니다 (템플릿)

6. 시험 (`tests/test_dashboard_service.py`: 4개를 추가해서 모두 16개)

시험	확인한 것
프로젝트가 없음	`[]`를 돌려줍니다
범위, 순서, 최대 3개	30일 전에 연 프로젝트, 삭제한 프로젝트, 남의 프로젝트는 빠지고, 다른 멤버만 연 공유 프로젝트는 포함되어 `New 0, Shared, New 1`이 나옵니다
응답 칸	bpm `96.00`, 박자 `7/8`, 생성 시각이 들어 있습니다
쿼리 수	SELECT가 1번만 나가고 `EXISTS`가 들어 있습니다

전체 시험 512개가 통과했고, `ruff` 검사도 통과했습니다.